런타임 호스트 (Runtime Host)

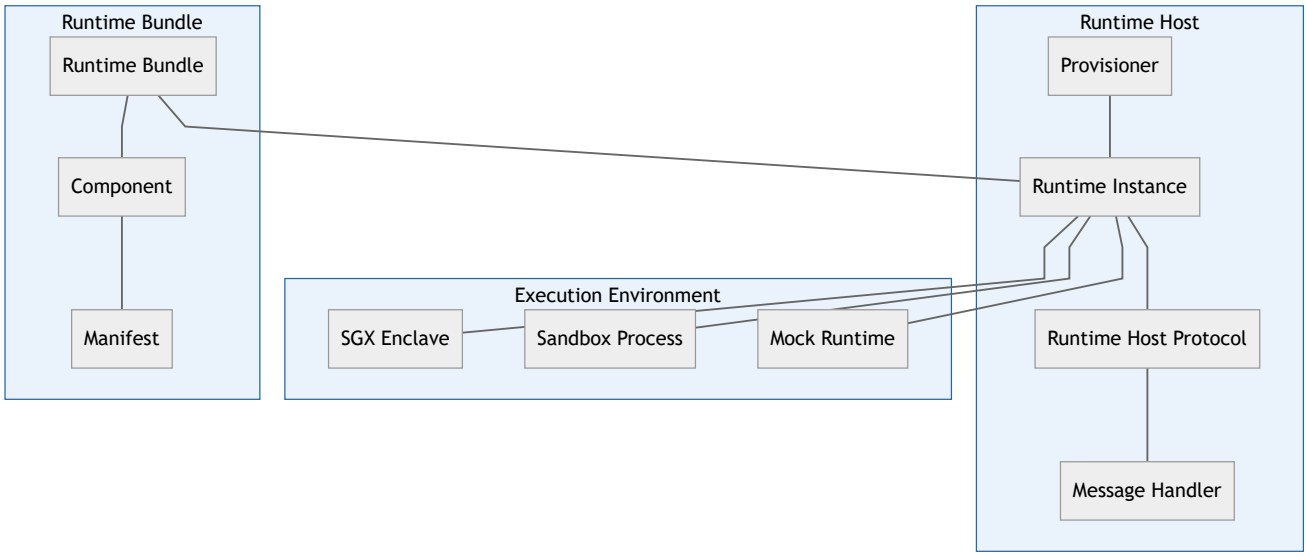
소개

런타임 호스트는 Oasis Core에서 런타임 컴포넌트를 실행하고 관리하는 핵심 시스템입니다. 런타임 호스트는 다양한 실행 환경(SGX, non-SGX)에서 런타임을 안전하게 격리하여 실행하고, Runtime Host Protocol(RHP)을 통해 런타임과 통신합니다.

런타임 호스트는 런타임의 생명주기를 관리하고, 보안 격리를 제공하며, 런타임과 합의 계층 간의 통신을 중개하는 역할을 담당합니다.

아키텍처

전체 구조



주요 컴포넌트

1. **Provisioner**: 런타임 인스턴스를 생성하고 관리
2. **Runtime Instance**: 실행 중인 런타임의 추상화
3. **Runtime Host Protocol**: 런타임과의 통신 프로토콜
4. **Execution Environment**: 실제 런타임 실행 환경

런타임 실행 환경

1. SGX 환경 (Intel SGX)

SGX 환경은 Intel SGX 기술을 사용하여 런타임을 안전한 엔클레이브 내에서 실행합니다.

특징

- **하드웨어 기반 보안**: Intel SGX 하드웨어 보안 기능 활용
- **메모리 암호화**: 엔클레이브 메모리는 하드웨어 수준에서 암호화
- **원격 증명**: 엔클레이브의 무결성을 원격으로 검증 가능
- **격리된 실행**: 호스트 OS로부터 완전히 격리된 실행 환경

SGX 런타임 구성 요소

```
type SGXMetadata struct {
// SGX 엔클레이브 실행 파일
Executable  string  `json:"executable"`
// SGX 엔클레이브 서명 파일
Signature  string  `json:"signature"`
```

```
}
```

SGX 디바이스 지원

```
// SGX 디바이스 자동 탐지
sgxDevices := []string{
"/dev/sgx_enclave", // 최신 드라이버
"/dev/sgx/enclave", // 구형 드라이버
"/dev/sgx", // 레거시 드라이버
"/dev/isgx"  // Intel 드라이버
}
```

2. Sandbox 환경 (Non-SGX)

Sandbox 환경은 일반적인 프로세스 격리를 사용하여 런타임을 실행합니다.

특징

- **프로세스 격리**: OS 수준의 프로세스 격리 활용
- **네임스페이스 분리**: Linux 네임스페이스를 통한 리소스 격리
- **파일 시스템 격리**: chroot 또는 컨테이너 기반 파일 시스템 격리
- **개발 친화적**: 디버깅과 개발이 용이

ELF 메타데이터

```
type ELFMetadata struct {
// ELF 실행 파일 이름
```

```
Executable string `json:"executable"`
}
```

3. Mock 환경 (테스트용)

Mock 환경은 테스트와 개발을 위한 시뮬레이션 환경입니다.

특징

- **SGX 시뮬레이션**: SGX 없이도 SGX 런타임 테스트 가능
- **빠른 개발**: 하드웨어 요구사항 없이 개발 가능
- **테스트 최적화**: 자동화된 테스트에 적합

런타임 호스트 프로토콜 (RHP)

프로토콜 개요

Runtime Host Protocol은 런타임 호스트와 런타임 간의 통신을 위한 JSON 기반 메시지 프로토콜입니다.

메시지 구조

```
type Message struct {
ID uint64 `json:"id"`
MessageType MessageType `json:"message_type"`
Body Body `json:"body"`
}
```

```go
type MessageType uint8

const (

MessageRequest MessageType = 1

MessageResponse MessageType = 2

)
```

주요 메시지 타입

1. 런타임 정보 요청/응답

```go
// 런타임 정보 요청

type RuntimeInfoRequest struct {

RuntimeID common.Namespace `json:"runtime_id"`

ConsensusBackend  string  `json:"consensus_backend"`

ConsensusProtocolVersion version.Version `json:"consensus_protocol_version"`

ConsensusChainContext  string  `json:"consensus_chain_context"`

LocalConfig  map[string]any `json:"local_config,omitempty"`

}


// 런타임 정보 응답

type RuntimeInfoResponse struct {

ProtocolVersion version.Version `json:"protocol_version"`

RuntimeVersion version.Version `json:"runtime_version"`

Features Features `json:"features,omitempty"`
```

```
}
```

2. 트랜잭션 실행 요청/응답

```
// 트랜잭션 배치 실행 요청
type RuntimeExecuteTxBatchRequest struct {
Mode ExecutionMode `json:"mode,omitempty"`
ConsensusBlock consensus.LightBlock `json:"consensus_block"`
RoundResults *roothash.RoundResults `json:"round_results"`
IORoot hash.Hash `json:"io_root"`
Inputs transaction.RawBatch `json:"inputs"`
InMessages []*message.IncomingMessage `json:"in_msgs,omitempty"`
Block block.Block `json:"block"`
Epoch beacon.EpochTime `json:"epoch"`
MaxMessages  uint32  `json:"max_messages"`
}

// 트랜잭션 배치 실행 응답
type RuntimeExecuteTxBatchResponse struct {
Batch ComputedBatch `json:"batch"`
TxHashes []hash.Hash `json:"tx_hashes,omitempty"`
TxRejectHashes []hash.Hash `json:"tx_reject_hashes,omitempty"`
TxInputRoot hash.Hash `json:"tx_input_root,omitempty"`
TxInputWriteLog storage.WriteLog `json:"tx_input_write_log,omitempty"`
```

```
}
```

3. TEE 관련 메시지

```
// TEE 능력 초기화 요청
type RuntimeCapabilityTEERakInitRequest struct {
TargetInfo []byte  `json:"target_info"`
}

// TEE 보고서 응답
type RuntimeCapabilityTEERakReportResponse struct {
RakPub signature.PublicKey `json:"rak_pub"`
RekPub *x25519.PublicKey `json:"rek_pub,omitempty"`
Report []byte  `json:"report"`
Nonce  string  `json:"nonce"`
}
```

프로토콜 연결 관리

연결 상태

```
type state uint8
const (
stateUninitialized state = iota
```

```
stateInitializing

stateReady

stateClosed

)
```

연결 초기화

```
// 호스트 모드 초기화

func (c *connection) InitHost(ctx context.Context, conn net.Conn, hi *HostInfo)

c.initConn(conn)

// 런타임 정보 요청

rsp, err := c.call(ctx, &Body{RuntimeInfoRequest: &RuntimeInfoRequest{

RuntimeID: c.runtimeID,

ConsensusBackend: hi.ConsensusBackend,

ConsensusProtocolVersion: hi.ConsensusProtocolVersion,

ConsensusChainContext: hi.ConsensusChainContext,

LocalConfig: hi.LocalConfig,

}})

// 프로토콜 버전 검증

if  info.ProtocolVersion.Major != version.RuntimeHostProtocol.Major {

return  nil, fmt.Errorf("incompatible protocol version")

}

return &rtVersion, nil
```

```
}
```

런타임 생명주기 관리

런타임 프로비저닝

1. 런타임 생성

```
type Provisioner interface {
// 새로운 런타임 인스턴스 생성
NewRuntime(cfg Config) (Runtime, error)
// 프로비저너 이름 반환
Name() string
}
```

2. 런타임 설정

```
type Config struct {
Name  string  // 런타임 이름
ID common.Namespace // 런타임 식별자
Component *bundle.ExplodedComponent // 런타임 컴포넌트
Extra any // 프로비저너별 설정
MessageHandler RuntimeHandler // 메시지 핸들러
LocalConfig  map[string]any // 로컬 설정
```

```
Log *log.Log // 로그 핸들

}
```

런타임 인스턴스 관리

런타임 인터페이스

```
type Runtime interface {

// 런타임 식별자

ID() common.Namespace

// 활성 버전 조회

GetActiveVersion() (*version.Version, error)

// 런타임 정보 조회

GetInfo(ctx context.Context) (*protocol.RuntimeInfoResponse, error)

// TEE 능력 조회

GetCapabilityTEE() (*node.CapabilityTEE, error)

// 런타임 호출

Call(ctx context.Context, body *protocol.Body) (*protocol.Body, error)

// 런타임 시작

Start()

// 런타임 중단

Abort(ctx context.Context, force  bool) error

// 런타임 정지

Stop()
```

```
// 이벤트 구독
WatchEvents() (<-chan *Event, pubsub.ClosableSubscription)
}
```

런타임 이벤트

```
type Event struct {
Started *StartedEvent // 시작 이벤트
FailedToStart *FailedToStartEvent // 시작 실패 이벤트
Stopped *StoppedEvent // 정지 이벤트
Updated *UpdatedEvent // 업데이트 이벤트
ConfigUpdated *ConfigUpdatedEvent // 설정 업데이트 이벤트
}

type StartedEvent struct {
Version version.Version // 런타임 버전
CapabilityTEE *node.CapabilityTEE // TEE 능력
}
```

보안 모델

SGX 보안 기능

1. 원격 증명 (Remote Attestation)

```go
// 원격 증명 프로세스
func (p *sgxProvisioner) updateCapabilityTEE(ctx context.Context, ts *teeState,
// RAK 보고서 요청
rakQuoteRes, err := conn.Call(ctx, &protocol.Body{
RuntimeCapabilityTEERakReportRequest: &protocol.Empty{},
})
// 증명 업데이트
attestation, err := ts.update(ctx, p, conn, report, nonce)
// TEE 능력 생성
capabilityTEE := &node.CapabilityTEE{
Hardware: node.TEEHardwareIntelSGX,
RAK: rakPub,
REK: rekPub,
Attestation: attestation,
}
return  capabilityTEE, nil
}
```

2. 엔클레이브 검증

```go
// 엔클레이브 바이너리 로딩 및 검증
func (p *sgxProvisioner) loadEnclaveBinaries(comp *bundle.ExplodedComponent) ([]
// SGXS 파일 로딩
```

```go
sgxs, err := os.ReadFile(sgxExecutablePath)
if err != nil {
return nil, nil, fmt.Errorf("failed to load enclave: %w", err)
}
// 엔클레이브 해시 계산
if err = enclaveHash.FromSgxsBytes(sgxs); err != nil {
return nil, nil, fmt.Errorf("failed to derive EnclaveHash: %w", err)
}
// 서명 검증
_, parsed, err := sigstruct.Verify(sig)
if err != nil {
return nil, nil, fmt.Errorf("failed to validate SIGSTRUCT: %w", err)
}
// 엔클레이브/서명 일치 확인
if parsed.EnclaveHash != enclaveHash {
return nil, nil, fmt.Errorf("enclave/SIGSTRUCT mismatch")
}
return sgxs, sig, nil
}
```

Sandbox 보안 기능

1. 프로세스 격리

```go
// Sandbox 설정
type Config struct {
Path  string  // 실행 파일 경로
Args []string  // 실행 인자
BindRW  map[string]string  // 읽기/쓰기 바인드 마운트
BindDev  map[string]string  // 디바이스 바인드 마운트
BindData  map[string]io.Reader // 데이터 바인드 마운트
SandboxBinaryPath  string  // Sandbox 바이너리 경로
AllowNetwork  bool  // 네트워크 접근 허용
}
```

2. 리소스 제한

- CPU 사용량 제한
- 메모리 사용량 제한
- 파일 시스템 접근 제한
- 네트워크 접근 제어

API 및 인터페이스

런타임 호스트 API

1. 런타임 생성

```go
// 새로운 런타임 생성
```

```go
runtime, err := provisioner.NewRuntime(host.Config{
Name: "my-runtime",
ID: runtimeID,
Component: component,
MessageHandler: handler,
LocalConfig: config,
Log: logger,
})
```

2. 런타임 시작

```go
// 런타임 시작
runtime.Start()

// 이벤트 구독
events, sub := runtime.WatchEvents()
defer sub.Close()

for event := range events {
switch {
case event.Started != nil:
fmt.Printf("Runtime started: %v\n", event.Started.Version)
case event.FailedToStart != nil:
fmt.Printf("Runtime failed to start: %v\n", event.FailedToStart.Error)
```

```
}
}
```

3. 런타임 호출

```
// 런타임에 메시지 전송
response, err := runtime.Call(ctx, &protocol.Body{
RuntimeExecuteTxBatchRequest: &protocol.RuntimeExecuteTxBatchRequest{
Mode: protocol.ExecutionModeExecute,
ConsensusBlock: consensusBlock,
Inputs: txBatch,
Block: block,
Epoch: epoch,
MaxMessages: maxMessages,
},
})

if err != nil {
return fmt.Errorf("runtime call failed: %w", err)
}

// 응답 처리
if response.RuntimeExecuteTxBatchResponse != nil {
```

```
batch := response.RuntimeExecuteTxBatchResponse.Batch

// 배치 처리 결과 사용

}
```

메시지 핸들러 구현

런타임 핸들러 인터페이스

```
type RuntimeHandler interface {

protocol.Handler

// 서브 핸들러 생성

NewSubHandler(comp *bundle.ExplodedComponent) (RuntimeHandler, error)

// 런타임 연결

AttachRuntime(id component.ID, host Runtime) error

}
```

핸들러 구현 예제

```
type myRuntimeHandler struct {

// 핸들러 상태

}


func (h *myRuntimeHandler) Handle(ctx context.Context, body *protocol.Body) (*pr

switch {

case  body.HostRPCCallRequest != nil:
```

```
// RPC 호출 처리
return h.handleRPCCall(ctx, body.HostRPCCallRequest)
case body.HostStorageSyncRequest != nil:
// 스토리지 동기화 처리
return h.handleStorageSync(ctx, body.HostStorageSyncRequest)
default:
return nil, fmt.Errorf("unsupported message type")
}
}
```

설정 및 사용 예제

SGX 런타임 설정

1. SGX 프로비저너 생성

```
sgxProvisioner, err := sgx.NewProvisioner(sgx.Config{
HostInfo: &protocol.HostInfo{
ConsensusBackend: "tendermint",
ConsensusProtocolVersion: version.ConsensusProtocol,
ConsensusChainContext: "oasis-core",
LocalConfig: localConfig,
},
```

```
CommonStore: store,
LoaderPath: "/opt/oasis/bin/oasis-core-runtime-loader",
IAS: iasEndpoints,
PCS: pcsService,
Consensus: consensus,
Identity: identity,
RuntimeAttestInterval: 2 * time.Hour,
SandboxBinaryPath: "/opt/oasis/bin/bwrap",
InsecureNoSandbox: false,
InsecureMock: false,
})
```

2. SGX 런타임 실행

```
// 런타임 컴포넌트 로딩
bundle, err := bundle.Open("runtime.orc")
if err != nil {
return fmt.Errorf("failed to open bundle: %w", err)
}
defer bundle.Close()

component, err := bundle.ExplodeComponent("ronl", dataDir)
if err != nil {
return fmt.Errorf("failed to explode component: %w", err)
```

```
}

// 런타임 생성 및 시작
runtime, err := sgxProvisioner.NewRuntime(host.Config{
Name: "sgx-runtime",
ID: runtimeID,
Component: component,
MessageHandler: handler,
LocalConfig: localConfig,
Log: logger,
})
if  err != nil {
return  fmt.Errorf("failed to create runtime: %w", err)
}

runtime.Start()
```

Sandbox 런타임 설정

1. Sandbox 프로비저너 생성

```
sandboxProvisioner, err := sandbox.NewProvisioner(sandbox.Config{
HostInfo: &protocol.HostInfo{
ConsensusBackend: "tendermint",
```

```
ConsensusProtocolVersion: version.ConsensusProtocol,
ConsensusChainContext: "oasis-core",
LocalConfig: localConfig,
},
SandboxBinaryPath: "/usr/bin/bwrap",
InsecureNoSandbox: false,
Logger: logger,
})
```

2. Sandbox 런타임 실행

```
// ELF 런타임 실행
runtime, err := sandboxProvisioner.NewRuntime(host.Config{
Name: "sandbox-runtime",
ID: runtimeID,
Component: component,
MessageHandler: handler,
LocalConfig: localConfig,
Log: logger,
})
if err != nil {
return fmt.Errorf("failed to create runtime: %w", err)
}
```

```
runtime.Start()
```

모니터링 및 디버깅

런타임 상태 모니터링

1. 런타임 정보 조회

```
// 런타임 버전 확인
version, err := runtime.GetActiveVersion()
if err != nil {
log.Printf("Failed to get runtime version: %v", err)
} else {
log.Printf("Runtime version: %v", version)
}

// 런타임 정보 조회
info, err := runtime.GetInfo(ctx)
if err != nil {
log.Printf("Failed to get runtime info: %v", err)
} else {
log.Printf("Runtime protocol version: %v", info.ProtocolVersion)
log.Printf("Runtime features: %+v", info.Features)
```

```
}
```

2. TEE 능력 확인

```
// TEE 능력 조회
capabilityTEE, err := runtime.GetCapabilityTEE()
if err != nil {
log.Printf("Failed to get TEE capability: %v", err)
} else if capabilityTEE != nil {
log.Printf("TEE Hardware: %v", capabilityTEE.Hardware)
log.Printf("RAK: %v", capabilityTEE.RAK)
if capabilityTEE.REK != nil {
log.Printf("REK: %v", capabilityTEE.REK)
}
}
```

로그 및 디버깅

1. 런타임 로그 래퍼

```
// 런타임 로그 래퍼 생성
logWrapper := host.NewRuntimeLogWrapper(
logger,
cfg.Log.Logger(),
"runtime_id", cfg.ID,
```

```
"runtime_name", cfg.Name,
"component", cfg.Component.ID(),
"provisioner", "sgx",
)
```

2. 디버그 모드 설정

```
// 디버그 모드에서 더미 서명 생성
if cmdFlags.DebugDontBlameOasis() {
logger.Warn("generating dummy enclave SIGSTRUCT",
"enclave_hash", enclaveHash,
)
sig, err = sigstruct.UnsafeDebugForEnclave(sgxs)
}
```

문제 해결

일반적인 문제

1. SGX 디바이스 없음

```
# SGX 디바이스 확인
ls -la /dev/sgx* /dev/isgx
```

```
# SGX 드라이버 설치 확인
lsmod | grep sgx
```

2. 런타임 시작 실패

```
// 런타임 이벤트 모니터링
events, sub := runtime.WatchEvents()
defer sub.Close()

for event := range events {
if event.FailedToStart != nil {
log.Printf("Runtime failed to start: %v", event.FailedToStart.Error)
// 실패 원인 분석 및 대응
}
}
```

3. 프로토콜 버전 불일치

```
// 프로토콜 버전 확인
if info.ProtocolVersion.Major != version.RuntimeHostProtocol.Major {
return fmt.Errorf("incompatible protocol version (expected: %s got: %s)",
version.RuntimeHostProtocol,
info.ProtocolVersion,
)
```

```
}
```

성능 최적화

1. 런타임 재사용

- 런타임 인스턴스를 재사용하여 초기화 오버헤드 감소
- 연결 풀링을 통한 통신 최적화

2. 메모리 관리

- 런타임 메모리 사용량 모니터링
- 가비지 컬렉션 최적화

3. 병렬 처리

- 여러 런타임 인스턴스 병렬 실행
- 로드 밸런싱을 통한 성능 향상

런타임 호스트는 Oasis Core의 런타임 실행 환경을 제공하는 핵심 컴포넌트로, 다양한 TEE 환경에서 안전하고 효율적인 런타임 실행을 보장합니다.